

Prelucrări ale datelor numerice — Rezolvări și Teorie

Clasa a IX-a · Profil Matematică-Informatică · Informatică Intensiv

Teorie completă — Prelucrări ale datelor numerice

1. Operații cu cifrele unui număr

Orice număr natural se poate descompune cifră cu cifră prin împărțiri/înmulțiri repetate cu 10.

EXTRAGERE CIFRE

Ultima cifră: `n % 10`

Eliminare ultimă: `n = n // 10`

Parcursire toate cifrele: buclă `while n > 0`

CONSTRUIRE NUMĂR

Adaugă cifra c la dreapta: `n = n * 10 + c`

Adaugă cifra c la stânga: `n = c * 10**k + n` (k = nr. cifre existente)

Oglindit: adaugă cifrele la dreapta în ordinea extragerii

2. Divizori și numere prime

Definiție: d este divizor al lui n dacă `n % d == 0`.

Optimizare cheie: dacă d divide n și $d \leq \sqrt{n}$, atunci $n/d \geq \sqrt{n}$ este și el divizor. Deci parcurgem doar până la $\sqrt{n}$ și adăugăm ambii divizori (d și n/d) în același pas.

Număr prim: are exact 2 divizori. Verificare eficientă: niciun d din $[2, \sqrt{n}]$ nu divide n.

Număr perfect: suma divizorilor proprii (fără n însuși) = n. Exemple: 6, 28, 496.

3. Algoritmul lui Euclid — CMMDC

Cu scăderi repetate: înlocuim (a, b) cu (a-b, b) dacă $a > b$ sau (a, b-a) dacă $b > a$, până $a=b$.

Cu împărțiri repetate (eficient): înlocuim (a, b) cu (b, $a \% b$) până $b=0$. La final $a = \text{CMMDC}$.

CMMDC: `a * b // cmmdc(a, b)` — folosim împărțirea înaintea înmulțirii pentru a evita overflow.

Simplificare fracție: împărțim numărătorul și numitorul la CMMDC.

4. Conversii între baze de numerație

Baza 10 → Baza 2 (sau orice bază b): împărțiri repetate la b; resturile citite de jos în sus formează reprezentarea.

Baza 2 → Baza 10: $\sum \text{cifra}_i \cdot 2^i$, unde i = poziția (de la dreapta, pornind de la 0).

Python util: `bin(n)` → reprezentare binară ca string; `int("101", 2)` → conversie bin → dec.

5. Exponențierea rapidă

Problema: calculul $b^e \% m$ pentru e foarte mare (ex. criptografie RSA).

Idee: dacă e par → $b^e = (b^{e/2})^2$; dacă e impar → $b^e = b \cdot b^{(e-1)}$. Se reduce e la jumătate la fiecare pas → $O(\log e)$ înmulțiri față de $O(e)$ clasic.

Exemplu: $2^{10} \rightarrow 2^5 \rightarrow 2^4 \rightarrow 2^2 \rightarrow 2^1 \rightarrow$ gata în 4 pași, nu 10.

6. Greșeli frecvente ale elevilor

- **Confuzie / vs //** — / dă float (3.0), // dă int (3). La operații cu cifre e nevoie de //.
- **Condiția de oprire a buclei** — `while n > 0` nu tratează $n=0$ inițial (care are o cifră: 0).
- **== vs = în condiție** — `if suma = n` e eroare de sintaxă; corect: `if suma == n`.
- **Numărul perfect** — suma include sau nu include n însuși? Prin definiție, *nu* include n → bucla merge până la n (exclusiv) sau până la $n/2$.

- **Parcurgerea divisorilor** — `range(1, n+1)` vs `range(1, int(n**0.5)+1)` ; diferența de performanță e uriașă pentru n mare.

SIMPLU Exercițiile 1–4 · 5 p / exercițiu

EXERCIȚIUL 1 (5P)

Urmărire algoritm pe $n = 3752$ — completare tabel.

● REZOLVARE — TABEL COMPLET

Pas	n (înainte)	n % 10	suma	n (după)
Start	3752	—	0	—
1	3752	2	2	375
2	375	5	7	37
3	37	7	14	3
4	3	3	17	0

OUTPUT

17

Ce calculează: suma cifrelor lui n . $3+7+5+2 = 17$.

Mecanism: `n % 10` extrage ultima cifră; `n // 10` elimină ultima cifră (echivalent cu „tăiem” cifra din dreapta).
Bucula se repetă până nu mai sunt cifre (n devine 0).

Extindere: Cum am modifica algoritmul pentru a calcula produsul cifrelor? (Răspuns: inițializăm `produs = 1` și înlocuim adunarea cu înmulțirea.)

EXERCIȚIUL 2 (5P)

Algoritmul lui Euclid cu împărțiri repetate pentru `cmmdc(48, 18)`.

● REZOLVARE — TABEL COMPLET

Pas	a	b	a % b
Start	48	18	12
1	18	12	6
2	12	6	0
3 (stop)	6	0	$b=0$, stop

CMMDC(48, 18) = 6. La fiecare pas: (a, b) devine $(b, a \% b)$. Oprim când $b=0$, iar a este rezultatul.

Verificare: $48 = 6 \times 8$ ✓ $18 = 6 \times 3$ ✓

De ce funcționează? $\text{CMMDC}(a, b) = \text{CMMDC}(b, a \bmod b)$ — proprietate matematică: orice divizor comun al lui a și b divide și $a \bmod b$, și invers.

EXERCIȚIUL 3 (5P)

Conversia 45 din baza 10 în baza 2.

● REZOLVARE — TABEL COMPLET

Împărțire	Câtul	Restul (cifră binară)
$45 \div 2$	22	1
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

REZULTAT (RESTURILE DE JOS ÎN SUS)

$45_{10} = 101101_2$

Verificare: $1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 0 + 8 + 4 + 0 + 1 = 45$ ✓

Python rapid: `bin(45)` → `'0b101101'` (prefixul 0b indică baza 2).

EXERCITIUL 4 (5P)

Asociere expresii Python cu semnificațiile lor.

● REZOLVARE

1 → E: `n % d == 0` verifică dacă d divide n (restul împărțirii este 0).

2 → A: `n % 10` returnează restul împărțirii la 10 = ultima cifră.

3 → B: `n // 10` returnează câtul împărțirii întregi la 10 = numărul fără ultima cifră.

4 → D: `n * 10 + c` deplasează cifrele existente la stânga și adaugă c ca nouă ultimă cifră.

5 → C: `n % 2 == 0` verifică dacă restul împărțirii la 2 este 0 → număr par.

MEDIU Exercițiile 5–8 · 10 p / exercițiu

EXERCITIUL 5 (10P)

Extrage și afișează cifrele pare ale lui n în ordinea în care apar.

● REZOLVARE

```
n = int(input("n = "))
original = n          # păstrăm n original pentru afișare

# Construim numărul din cifrele pare, în ordine
# Problemă: extragerea cifrelor le dă în ordine INVERSĂ
# Soluție: mai întâi colectăm cifrele, apoi le procesăm invers

cifre = []
temp = n
while temp > 0:
    cifre.append(temp % 10)
    temp = temp // 10
```

```

cifre.reverse()      # acum cifrele sunt în ordinea corectă (stânga→dreapta)

rezultat = ""
for c in cifre:
    if c % 2 == 0:
        rezultat += str(c)

if rezultat == "":
    print("Nicio cifră pară")
else:
    print(rezultat)

```

TESTE

n=3472 → 42 n=135 → Nicio cifră pară n=2468 → 2468

Problema ordinii: extragând cifrele prin `% 10` obținem cifra din dreapta întâi. Pentru a păstra ordinea originală, colectăm într-o listă și inversăm. Alternativ, se poate converti `n` la string: `str(n)` și itera direct.

EXERCITIUL 6 (10P)

Oglindit și verificare palindrom pentru `n` dat.

● REZOLVARE

```

n = int(input("n = "))
original = n
oglindit = 0

temp = n
while temp > 0:
    cifra = temp % 10
    oglindit = oglindit * 10 + cifra # adăugăm cifra la dreapta oglinditului
    temp = temp // 10

print(f"Oglindit: {oglindit}")
if original == oglindit:
    print("Palindrom: Da")
else:
    print("Palindrom: Nu")

```

TESTE

n=1234 → Oglindit: 4321 | Palindrom: Nu n=1221 → Oglindit: 1221 | Palindrom: Da n=12100 → Oglindit: 121 | Palindrom: Nu ← atenție! zerouri finale

Construcția oglinditului: la fiecare pas, "facem loc" pentru noua cifră: `oglindit * 10 + cifra`. Cifrele se adaugă în ordinea extragerii (dreapta → stânga din original = stânga → dreapta în oglindit).

Caz special: n=12100 are oglindit 121 (nu 00121), deoarece zerourile de la final devin zerouri de la început și sunt ignorate în reprezentarea numerică.

EXERCITIUL 7 (10P)

Toți divizorii lui `n`, cu optimizare până la $\sqrt{n}$.

● REZOLVARE

```
import math

n = int(input("n = "))
divizori = []

for d in range(1, int(math.sqrt(n)) + 1):
    if n % d == 0:
        divizori.append(d)          # divisorul mic
        if d != n // d:             # evităm dublarea când d = √n
            divizori.append(n // d) # perechea lui (divisorul mare)

divizori.sort()
print(f"Divizori: {' '.join(map(str, divizori))}")
print(f"Total: {len(divizori)} divizori")
```

TESTE

n=36 → Divizori: 1 2 3 4 6 9 12 18 36 | Total: 9 divizori n=17 → Divizori: 1 17 | Total: 2 divizori (număr prim) n=16 → Divizori: 1 2 4 8 16 | Total: 5 divizori

Optimizarea $\sqrt{n}$: dacă $d \mid n$ și $d \leq \sqrt{n}$, atunci $n/d \geq \sqrt{n}$ este și el divisor. Adăugăm ambii dintr-o dată. Condiția `d != n//d` previne dublarea lui $\sqrt{n}$ (ex: pentru $n=36$, $d=6 \rightarrow 36//6=6$, nu adăugăm de două ori).

Performanță: pentru $n=1.000.000$, parcurgem 1000 pași în loc de 1.000.000.

EXERCIȚIUL 8 (10P)

Identifică și corectează cele două greșeli din codul pentru număr perfect.

● REZOLVARE — IDENTIFICAREA GREȘELILOR

Greșeala 1 — logică: `range(1, n)`

Includem n însuși ca posibil divisor propriu? Nu! Dar `range(1, n)` merge până la $n-1$, deci nu include n — aceasta e *corectă* pentru suma divizorilor proprii. Totuși, mai eficient e `range(1, n//2 + 1)` deoarece niciun număr mai mare decât $n/2$ (în afară de n însuși) nu poate fi divisor al lui n .

Greșeala 2 — sintaxă FATALĂ: `if suma = n`

În Python, `=` este atribuire, nu comparare. Compararea corectă este `==`. Această greșeală provoacă `SyntaxError` și programul nu rulează deloc.

```
# Cod corectat:
n = int(input("n = "))
suma = 0
for d in range(1, n // 2 + 1): # optimizat: până la n//2
    if n % d == 0:
        suma = suma + d
if suma == n:                  # Fix: "=" nu "="
    print("Număr perfect")
else:
    print("Nu este perfect")
```

TESTE

n=6 → Număr perfect ($1+2+3=6$) n=28 → Număr perfect ($1+2+4+7+14=28$) n=12 → Nu este perfect ($1+2+3+4+6=16 \neq 12$)

EXERCIȚIUL 9 (15P)

CMMDC + CMMMC + simplificare fracție pentru $a=84$, $b=36$.

● REZOLVARE

```
def cmmdc(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def cmmmc(a, b):
    return a * b // cmmdc(a, b)  # // înainte de * evită numere mari

def simplifica(numarator, numitor):
    d = cmmdc(numarator, numitor)
    return numarator // d, numitor // d

a = int(input("a = "))
b = int(input("b = "))

d = cmmdc(a, b)
m = cmmmc(a, b)
nr, nd = simplifica(a, b)

print(f"CMMDC({a}, {b}) = {d}")
print(f"CMMMC({a}, {b}) = {m}")
print(f"Frația {a}/{b} simplificată = {nr}/{nd}")
```

OUTPUT PENTRU A=84, B=36

CMMDC(84, 36) = 12 CMMMC(84, 36) = 252 Frația 84/36 simplificată = 7/3

Verificare: $84 = 12 \times 7$ ✓ $36 = 12 \times 3$ ✓ $7/3$ nu se mai poate simplifica (CMMDC(7,3)=1).

CMMMC = $84 \times 36 / 12 = 3024 / 12 = 252$. Verificare: $252 = 84 \times 3 = 36 \times 7$ ✓

EXERCIȚIUL 10 (15P)

Descompunere în factori primi a lui n în forma $2^a \cdot 3^b \cdot \dots$

● REZOLVARE

```
n = int(input("n = "))
original = n
factori = {}  # dicționar: factor → putere

d = 2
while d * d <= n:  # parcurgem până la √n
    while n % d == 0:  # cât timp d divide n
        factori[d] = factori.get(d, 0) + 1
        n = n // d
    d += 1
if n > 1:  # restul rămas este un factor prim
```

```

factori[n] = factori.get(n, 0) + 1

if len(factori) == 1 and factori.get(original, 0) == 1:
    print(f"{original} este număr prim")
else:
    rezultat = " · ".join(f"{f}^{p}" for f, p in factori.items())
    print(rezultat)

```

TESTE

$n=360 \rightarrow 2^3 \cdot 3^2 \cdot 5^1$ $n=17 \rightarrow 17$ este număr prim $n=1024 \rightarrow 2^{10}$ $n=100 \rightarrow 2^2 \cdot 5^2$

Algoritm: împărțim n la $d=2,3,4,\dots$ cât timp $d^2 \leq n$. La fiecare d , împărțim repetat cât timp $d \mid n$ (eliminând toate aparițiile lui d). Dacă la final $n > 1$, înseamnă că n însuși este un factor prim (mai mare decât $\sqrt{n}$ original).

Eficiență: parcurgem până la $\sqrt{n}$, nu până la n .

EXERCIȚIUL 11 (15P)

Exponențiere rapidă: `putere_rapida(baza, exp, mod)` cu contorizare pași.

● REZOLVARE

```

def putere_rapida(baza, exp, mod):
    rezultat = 1
    baza = baza % mod
    pasi = 0

    while exp > 0:
        if exp % 2 == 1:          # exp impar: înmulțim rezultatul
            rezultat = (rezultat * baza) % mod
            pasi += 1
        exp = exp // 2           # înjumătățim exponentul
        baza = (baza * baza) % mod # ridicăm la pătrat baza
        pasi += 1

    return rezultat, pasi

def putere_clasica(baza, exp, mod):
    rezultat = 1
    for _ in range(exp):
        rezultat = (rezultat * baza) % mod
    return rezultat, exp # exp înmulțiri

for test in [(2, 10, 1000), (3, 100, 1000007)]:
    b, e, m = test
    r_rapid, p_rapid = putere_rapida(b, e, m)
    _, p_clasic = putere_clasica(b, e, m)
    print(f"{b}^{e} % {m} = {r_rapid}")
    print(f"  Rapid: {p_rapid} pași | Clasic: {p_clasic} pași\n")

```

OUTPUT

$2^{10} \% 1000 = 24$ Rapid: 8 pași | Clasic: 10 pași $3^{100} \% 1000007 = 981746$ Rapid: 13 pași | Clasic: 100 pași

Ideea cheie: la fiecare pas înjumătățim exponentul $\rightarrow O(\log_2 e)$ pași în loc de $O(e)$.

Pentru $e=100$: $\log_2(100) \approx 7$ pași teoretic (în practică ~ 13 datorită pașilor suplimentari pentru biți de 1).

Aplicație reală: RSA folosește exponențiere cu $e \approx 2^{2048}$ — imposibil clasic, fezabil rapid!

EXERCITIUL 12 (15P)

Problema semafoarelor: CMMC al ciclurilor, primele 5 momente de sincronizare.

● REZOLVARE

```
def cmmdc(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def cmmc(a, b):
    return a * b // cmmdc(a, b)

# Date problemă
ciclu_A = 45 + 30 # 75 secunde – durata unui ciclu complet al semaforului A
ciclu_B = 60 + 40 # 100 secunde – durata unui ciclu complet al semaforului B

# Primul moment de sincronizare = CMMC al ciclurilor
# De ce? Ambele trec pe roșu când ciclul lor se resetează simultan
# Asta se întâmplă exact la multiplii comuni ai celor două cicluri
sincronizare = cmmc(ciclu_A, ciclu_B)

print(f"Ciclu A: {ciclu_A}s | Ciclu B: {ciclu_B}s")
print(f"CMMDC({ciclu_A}, {ciclu_B}) = {cmmdc(ciclu_A, ciclu_B)}")
print(f"CMMC({ciclu_A}, {ciclu_B}) = {sincronizare}s\n")

print("Primele 5 momente de sincronizare:")
for i in range(1, 6):
    moment = sincronizare * i
    min_tot = moment // 60
    sec = moment % 60
    print(f" Momentul {i}: {moment}s = {min_tot} min {sec}s")
```

OUTPUT

Ciclu A: 75s | Ciclu B: 100s CMMDC(75, 100) = 25 CMMC(75, 100) = 300s Primele 5 momente de sincronizare: Momentul 1: 300s = 5 min 0s Momentul 2: 600s = 10 min 0s Momentul 3: 900s = 15 min 0s Momentul 4: 1200s = 20 min 0s Momentul 5: 1500s = 25 min 0s

Raționament matematic: ambele semafoare trec pe roșu simultan când timpul scurs este multiplu comun al ambelor cicluri. Cel mai mic astfel de moment este CMMC. Celelalte momente sunt multiplii lui CMMC.

Verificare: $300 / 75 = 4$ cicluri complete A ✓ $300 / 100 = 3$ cicluri complete B ✓